

Imagens das cartas: origem, cache e renderização

t4kena · análise do código-fonte (Next.js 16.3 + Supabase) · 24/09/2026

Em resumo: o site não consome nenhuma API de cartas. Os dados vêm do Supabase (Postgres). As imagens vêm da pasta `public/` do projeto (as cerca de 190 cartas avulsas e os selados) ou do bucket `product-images` do Supabase Storage (o que é cadastrado pelo painel). Tudo passa pelo `next/image`, com cache de dados no servidor (`unstable_cache`, 60 s) e cache HTTP das imagens no navegador (1 dia, mais 7 dias de `stale-while-revalidate`).

Tema	Situação
API externa de cartas	Não usa Supabase + arquivos locais
Cache de dados	Sim <code>unstable_cache</code> com tags, no servidor
Cache de imagens	Sim Cabeçalho <code>Cache-Control</code> em <code>/assets</code> + otimizador do Next
Skeleton / blur / fallback de erro	Não implementado
Lazy loading e prioridade	Sim
WebP / redimensionamento	Parcial WebP sim, AVIF não
Prevenção de CLS	Sim
Virtualização	Não implementado Paginação no servidor

1. Fonte de dados e integração

1.1 Qual API ou serviço é consumido

Nenhuma API externa de cartas O código não chama PokéAPI, pokemontcg.io, TCGdex nem outra fonte parecida. Os metadados (nome, preço, estoque, caminho da imagem) ficam nas tabelas `products`, `product_variants` e `product_images` do Supabase.

O campo `storage_path` define de onde vem o arquivo:

`src/lib/store/supabase-mapping.ts:74`

```
export function resolveImageUrl(storagePath: string, supabaseUrl: string) {
  if (!storagePath) return "";
  if (storagePath.startsWith("/") || storagePath.startsWith("http")) return storagePath;
  return `${supabaseUrl}/storage/v1/object/public/product-images/${storagePath}`;
}
```

- **Caminhos com `/`** (ex.: `/assets/cartas/umbreon-ex-b654cc0ece.jpg`) são arquivos da pasta `public/` que a Vercel serve junto com o site. É o caso das cerca de 190 cartas avulsas, importadas da vitrine da MYP Cards (commit `6af888c`).
- **O resto** vira URL pública do bucket `product-images` do Supabase Storage. São as fotos enviadas pelo painel (`src/components/painel/campo-de-imagem.tsx`).

As artes oficiais da Celebração de 30 Anos foram baixadas *uma vez* do site da Pokémon por `scripts/sync-thirty-official.mjs` e `scripts/sync-thirty-showcase.mjs`. Com o site no ar, nada é consultado lá.

1.2 Como e onde as requisições são disparadas

Sempre no **servidor**, dentro dos Server Components. Uma fachada escolhe entre o Supabase e o arquivo `data/store.json`:

`src/lib/store/repository.ts:17`

```
function impl() {
  return usesSupabaseCatalog() ? supabaseRepository : fileRepository;
}
```

`src/lib/store/supabase-repository.ts:57`

```
const readPublishedProducts = unstable_cache(async () => {
  const supabase = createPublicClient();
  const rows = await withTransientReadRetry(async () => unwrap(
    await supabase.from("products").select(PRODUCT_SELECT)
      .eq("status", "published").order("created_at", { ascending: false }),
    "Listar catálogo"));
  return toStoreProducts(rows).filter((product) => product.image !== "");
}, ...);
```

O navegador não faz nenhum fetch de catálogo. Ele recebe o HTML pronto com as tags de imagem.

2. Estratégia e escopo de cache

2.1 Mecanismos em uso

Dados: `unstable_cache` do Next.js, com revalidação por tempo e por tag.

`src/lib/store/supabase-repository.ts:71–73 e 108–117`

```
}, ["storefront-published-products-v1"], {
  revalidate: 60,
  tags: [STOREFRONT_PRODUCTS_CACHE_TAG],
});
// configurações da loja: revalidate: 300, tags: [STOREFRONT_SETTINGS_CACHE_TAG]
```

Quando o painel salva, o cache é invalidado na hora:

`src/app/admin/actions.ts:33`

```
function revalidateStorefront() {
  updateTag(STOREFRONT_PRODUCTS_CACHE_TAG);
  updateTag(STOREFRONT_SETTINGS_CACHE_TAG);
  revalidatePath("/", "layout");
}
```

Imagens: cabeçalho HTTP definido no `next.config.ts`. Sem ele, a Vercel entregaria `max-age=0` e a foto seria baixada de novo a cada visita.

`next.config.ts`

```
{
  source: "/assets/:path*",
  headers: [{ key: "Cache-Control",
    value: "public, max-age=86400, stale-while-revalidate=604800" }],
}
```

Não implementado React Query, SWR, Service Worker, Cache API e Redis.

2.2 Persistência entre rotas

O **cache é compartilhado**. Catálogo, home, coleções e página de produto leem a mesma entrada `storefront-published-products-v1`. Trocar de página não vai ao banco enquanto o cache estiver válido (60 s).

A página de produto é `force-dynamic`, então o HTML dela é refeito a cada visita, mas os dados continuam vindo do cache:

`src/app/(loja)/produto/[slug]/page.tsx:53`

```
export const dynamic = "force-dynamic";
```

Uma imagem baixada numa página fica no cache do navegador e não é baixada de novo nas outras.

2.3 Camada do cache

Híbrida:

- **Servidor:** dados no Data Cache do Next (Vercel) e versões otimizadas das imagens em `/_next/image`.
- **Cliente:** cache HTTP do navegador para as imagens, guiado pelo `Cache-Control`.

3. Arquitetura e componentes de renderização

3.1 Componente base

`next/image` em toda a loja. O único `<img>` nativo está nos anexos do chat do pedido (`src/components/painel/order-chat-messages.tsx:70`).

`src/components/loja/produto-card.tsx:37` — card do catálogo, das coleções e da linha de 30 anos

```
<Image
  src={product.image}
  alt={product.imageAlt}
  width={232}
  height={carta ? 325 : 232}
  sizes="(max-width: 620px) 45vw, 240px"
  priority={prioridade}
/>
```

Outros pontos que renderizam a carta: galeria da página de produto (`src/components/store/product-gallery.tsx`), carrinho (`carrinho/cart-client.tsx:110`), gaveta da sacola (`cart/cart-drawer.tsx:74`), checkout (`checkout/checkout-client.tsx:178`) e busca da navbar (`nav2/navbar.tsx:980`).

A galeria só coloca na página a foto atual e as duas vizinhas, e mantém montadas as que já apareceram:

`src/components/store/product-gallery.tsx`

```
const [montadas, setMontadas] = useState<Set<number>>(  
  () => new Set(shots.length > 1 ? [0, 1] : [0]),  
);
```

3.2 Loading, placeholder e erro

Não implementado

- Nenhuma imagem usa `placeholder="blur"`, `blurDataURL`, `onError` ou `onLoad`.
- Não existe `loading.tsx` nem `error.tsx` em `src/app`.
- O único "placeholder" é a cor de fundo da moldura, visível até a imagem chegar (`src/styles/loja.css:2085`: `background: #f2f7f0`).
- Produto sem imagem é tirado da lista antes de renderizar (`supabase-repository.ts:71`). Uma imagem que falha ao carregar não tem fallback.

4. Otimizações de performance e UX

4.1 Lazy loading e prioridade

Implementado As imagens do `next/image` carregam sob demanda por padrão. A prioridade é dada de forma explícita:

`src/app/(loja)/catalogo/page.tsx:157` — os 4 primeiros cards de cada página

```
<ProdutoCard product={product} key={product.id} prioridade={i < 4} />
```

`src/components/store/product-gallery.tsx` — a capa da galeria

```
priority={i === 0}  
sizes="(max-width: 600px) 88vw, 44vw"
```

A vitrine da home e as coleções usam `loading="lazy"` de forma explícita (`home/collector-shelf.tsx:109`, `colecoes/grade.tsx:126`). `fetchPriority` não é usado diretamente; o `priority` do `next/image` já faz esse papel.

4.2 Formatos modernos e redimensionamento

Parcial O otimizador do Next entrega **WebP** e gera larguras diferentes a partir do `sizes`. Só o Storage do Supabase é liberado como origem remota:

`next.config.ts`

```
images: {  
  remotePatterns: supabaseHost  
    ? [{ protocol: "https", hostname: supabaseHost, pathname: "/storage/v1/object/public/**" }]  
    : [],  
},
```

A configuração não tem `formats`, então o **AVIF não está ativo**. A prévia de upload do painel usa `unoptimized` (`campo-de-imagem.tsx:93`), o que é correto para uma prévia local.

4.3 Prevenção de CLS

Implementado Todo `<Image>` recebe `width` e `height`, e a moldura tem proporção fixa, com 3:4 para cartas:

src/styles/loja.css:2079–2096

```
.lj-produto-arte      { aspect-ratio: 1; }  
.lj-produto-arte--carta { aspect-ratio: 3 / 4; }  
.lj-produto-arte img   { width: 100%; height: 100%; object-fit: contain; }
```

4.4 Virtualização

Não implementado O volume é controlado por **paginação no servidor**: 16 itens por página no desktop e 6 no mobile. Os `IntersectionObserver` que existem servem só para animação.

src/app/(loja)/catalogo/page.tsx:28

```
{ pageSize: 16, className: styles.desktop },  
{ pageSize: 6,  className: styles.mobile },
```

5. Pontos de atenção

Ponto	Detalhe e sugestão
Duas grades renderizadas ao mesmo tempo no catálogo	<code>catalogo/page.tsx:149</code> renderiza a grade de desktop e a de mobile, e o CSS esconde uma delas. Como <code>prioridade={i < 4}</code> vale nas duas, podem ser pré-carregadas 8 imagens em vez de 4, incluindo as da grade escondida.
Nenhum fallback de erro	Uma foto apagada do bucket, ou um caminho errado, deixa o card com a moldura vazia. Um <code>onError</code> com imagem padrão resolve.
AVIF desligado	Dá para ligar com <code>images: { formats: ["image/avif", "image/webp"] }</code> . Os arquivos ficam menores, mas o primeiro acesso a cada imagem demora mais para processar.
Sem placeholder de carregamento	Só a cor de fundo aparece. Um <code>blurDataURL</code> ou um skeleton deixaria o carregamento mais suave nas conexões lentas.

Análise feita somente pela leitura do código-fonte do repositório local, sem medições em produção.